

THỰC HÀNH MÔN THỊ GIÁC MÁY TÍNH

BÀI TẬP THỰC HÀNH SỐ 4

Lab 4. VẼ ĐỐI TƯỢNG, PHÂN TÍCH CHUYỂN ĐỘNG

1. Vẽ đường thẳng

`cv2.line`

`line(img, pt1, pt2, color[, thickness[, lineType[, shift]]]) -> img`

- . @param img Image.
- . @param pt1 First point of the line segment.
- . @param pt2 Second point of the line segment.
- . @param color Line color.
- . @param thickness Line thickness.
- . @param lineType Type of the line. See #LineTypes.
- . @param shift Number of fractional bits in the point coordinates.

2. Vẽ hình chữ nhật

`cv2.rectangle`

`rectangle(img, pt1, pt2, color[, thickness[, lineType[, shift]]]) -> img`

- . @brief Draws a simple, thick, or filled up-right rectangle.
- . The function `cv::rectangle` draws a rectangle outline or a filled rectangle whose two opposite corners are `pt1` and `pt2`.
- . @param img Image.
- . @param pt1 Vertex of the rectangle.
- . @param pt2 Vertex of the rectangle opposite to `pt1`.
- . @param color Rectangle color or brightness (grayscale image).
- . @param thickness Thickness of lines that make up the rectangle. Negative values, like `#FILLED`, mean that the function has to draw a filled rectangle.
- . @param lineType Type of the line. See #LineTypes
- . @param shift Number of fractional bits in the point coordinates.

3. Vẽ hình tròn

`cv2.circle`

`circle(img, center, radius, color[, thickness[, lineType[, shift]]]) -> img`

- . @brief Draws a circle.

- . The function `cv::circle` draws a simple or filled circle with a given center and radius.
- . @param `img` Image where the circle is drawn.
- . @param `center` Center of the circle.
- . @param `radius` Radius of the circle.
- . @param `color` Circle color.
- . @param `thickness` Thickness of the circle outline, if positive. Negative values, like `#FILLED`, mean that a filled circle is to be drawn.
- . @param `lineType` Type of the circle boundary. See `#LineTypes`
- . @param `shift` Number of fractional bits in the coordinates of the center and in the radius value.

4. Vẽ ellipse

`cv2.ellipse`

`ellipse(img, center, axes, angle, startAngle, endAngle, color[, thickness[, lineType[, shift]]]) -> img`

- . @brief Draws a simple or thick elliptic arc or fills an ellipse sector.

- . @param `img` Image.
- . @param `center` Center of the ellipse.
- . @param `axes` Half of the size of the ellipse main axes.
- . @param `angle` Ellipse rotation angle in degrees.
- . @param `startAngle` Starting angle of the elliptic arc in degrees.
- . @param `endAngle` Ending angle of the elliptic arc in degrees.
- . @param `color` Ellipse color.
- . @param `thickness` Thickness of the ellipse arc outline, if positive. Otherwise, this indicates that a filled ellipse sector is to be drawn.
- . @param `lineType` Type of the ellipse boundary. See `#LineTypes`
- . @param `shift` Number of fractional bits in the coordinates of

5. Vẽ đa giác

`cv2.polylines`

`polylines(img, pts, isClosed, color[, thickness[, lineType[, shift]]]) -> img`

- . @brief Draws several polygonal curves.

- . @param `img` Image.
- . @param `pts` Array of polygonal curves.
- . @param `isClosed` Flag indicating whether the drawn polylines are closed or not. If they are closed, the function draws a line from the last vertex of each curve to its first vertex.
- . @param `color` Polyline color.
- . @param `thickness` Thickness of the polyline edges.
- . @param `lineType` Type of the line segments. See `#LineTypes`
- . @param `shift` Number of fractional bits in the vertex coordinates.

6. Chèn text vào ảnh

Các font:

```
FONT_HERSHEY_COMPLEX int
FONT_HERSHEY_COMPLEX_SMALL
FONT_HERSHEY_DUPLEX
FONT_HERSHEY_PLAIN
FONT_HERSHEY_SCRIPT_COMPLEX
FONT_HERSHEY_SCRIPT_SIMPLEX
FONT_HERSHEY_SIMPLEX
FONT_HERSHEY_TRIPLEX
FONT_ITALIC
```

cv2.putText

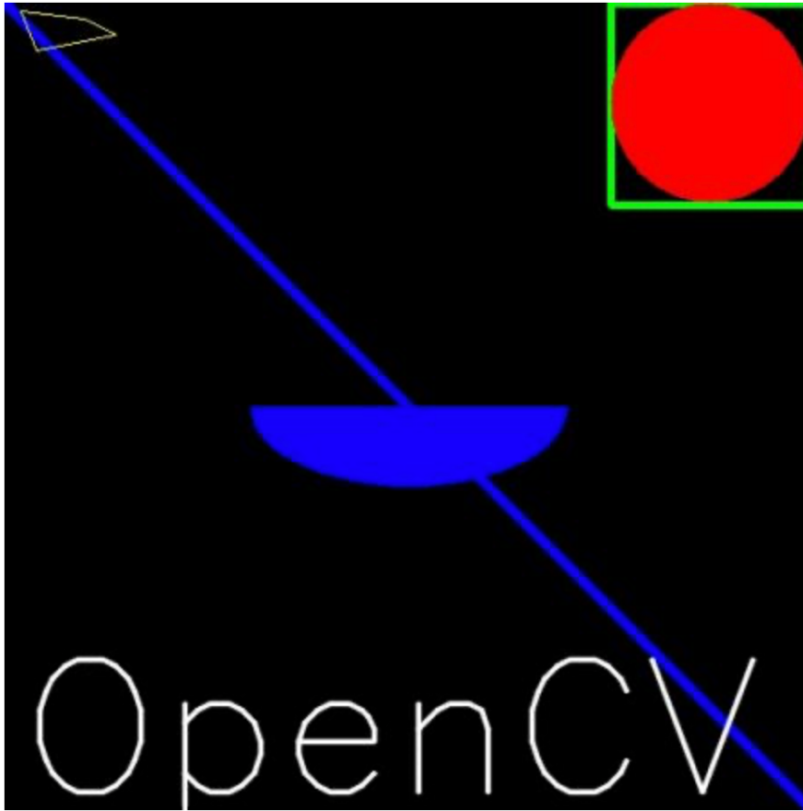
```
putText(img, text, org, fontFace, fontScale, color[, thickness[,
lineType[, bottomLeftOrigin]]]) -> img
. @brief Draws a text string.
```

```
. @param img Image.
. @param text Text string to be drawn.
. @param org Bottom-left corner of the text string in the image.
. @param fontFace Font type, see #HersheyFonts.
. @param fontScale Font scale factor that is multiplied by the
font-specific base size.
. @param color Text color.
. @param thickness Thickness of the lines used to draw a text.
. @param lineType Line type. See #LineTypes
. @param bottomLeftOrigin When true, the image data origin is
at the bottom-left corner. Otherwise,
. it is at the top-left corner.
```

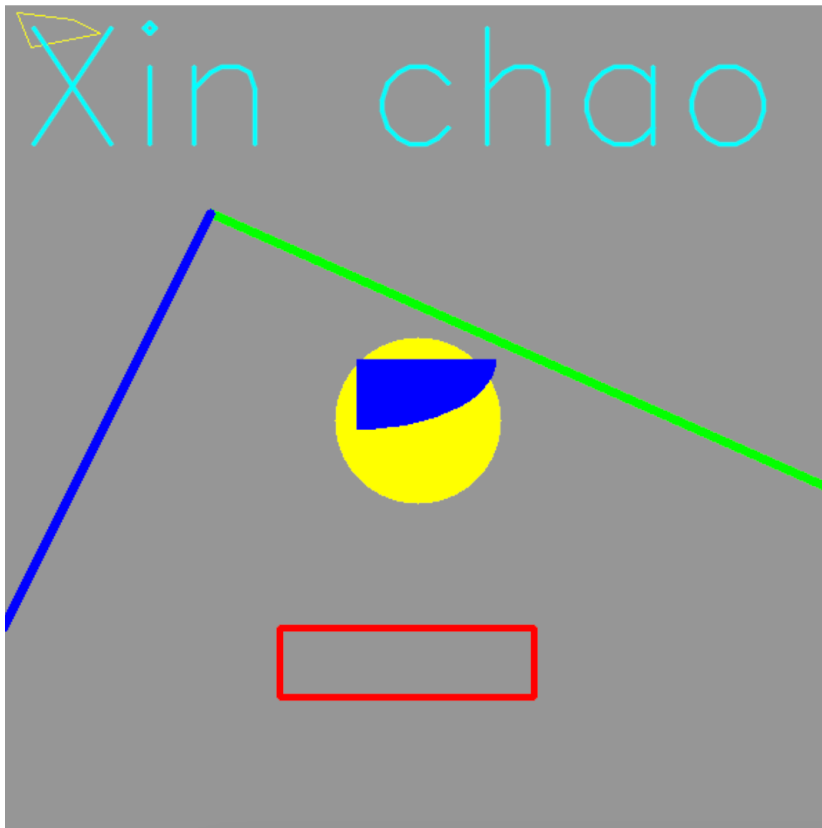
7. Thực hiện tạo hình ảnh như sau:

```
# Create a black image
img = np.zeros((512,512,3), np.uint8)

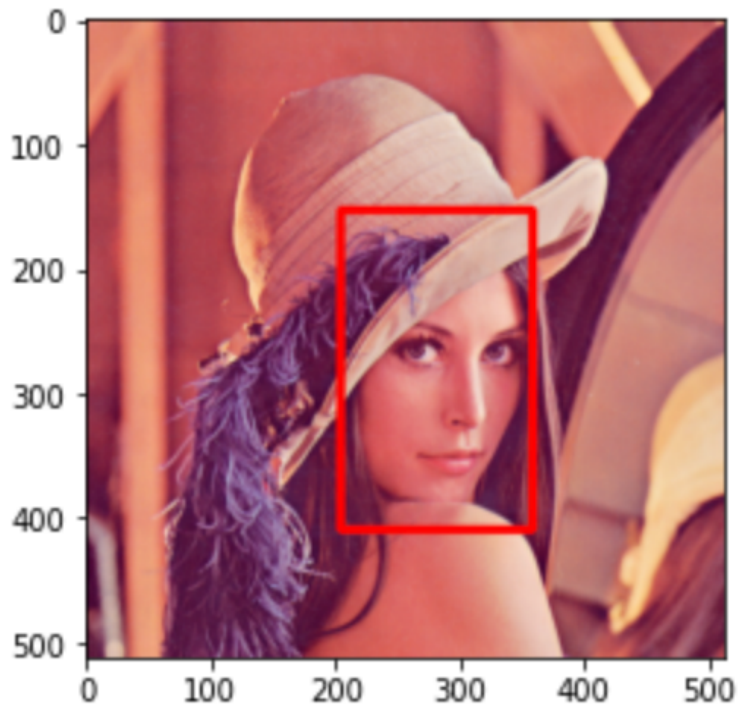
# Draw a diagonal blue line with thickness of 5 px
cv2.line(img,(0,0),(511,511),(255,0,0),5)
cv2.rectangle(img,(384,0),(510,128),(0,255,0),3)
cv2.circle(img,(447,63), 63, (0,0,255), -1)
cv2.ellipse(img,(256,256),(100,50),0,0,180,255,-1)
pts = np.array([[10,5],[20,30],[70,20],[50,10]], np.int32)
pts = pts.reshape((-1,1,2))
cv2.polylines(img,[pts],True,(0,255,255))
font = cv2.FONT_HERSHEY_SIMPLEX
cv2.putText(img,'OpenCV',(10,500), font, 4,(255,255,255),2,cv2.LINE_AA)
```



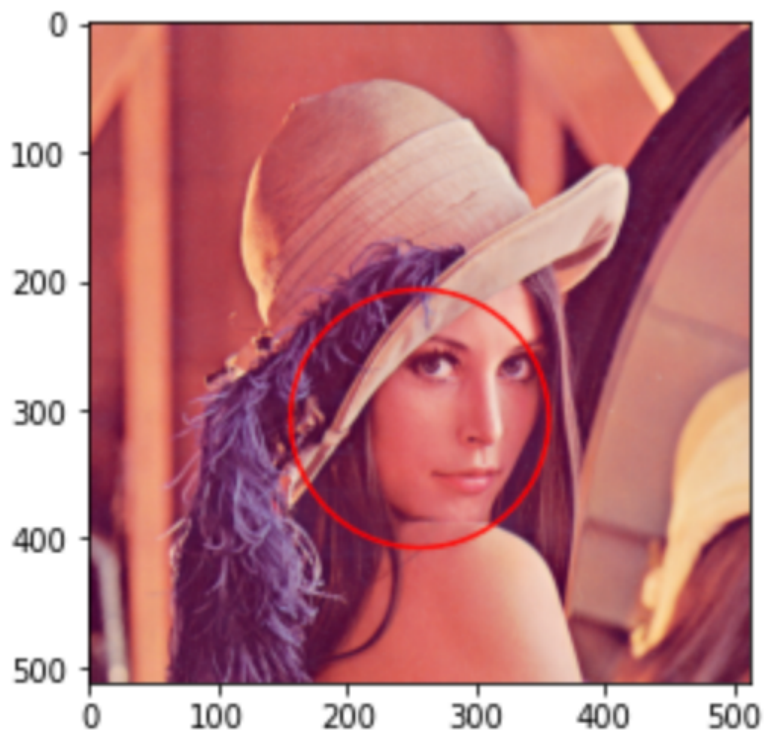
8. Thực hiện tạo hình ảnh như sau:



9. Xác định khuôn mặt trong ảnh bằng khung hình chữ nhật



10. Xác định khuôn mặt trong ảnh bằng khung hình tròn



- 11. Tiến hành khử nhiễu ảnh bằng phép biến đổi Wavelet và nhận dạng khuôn mặt bằng khung hình chữ nhật trên ảnh đã khử nhiễu.**
- 12. Thực hiện tách ra frame ảnh từ 1 video đã cho sẵn và lưu các frame ảnh đã tách vào google drives.**
- 13. Dùng biến đổi Wavelet khử nhiễu toàn bộ các frame ảnh đã tách ra được từ bài tập 12. Sau đó tìm hiểu đơn vị đánh giá mức độ nhiễu của ảnh (Peak-Signal-to-Noise-Ratio, Mean Square Error).**

=== HẾT ===